

CMSC 754: Lecture 3

Convex Hulls

Tuesday, Sep 9, 2014

Reading: Chapter 1 in the 4M's (de Berg, Cheong, van Kreveld, Schwarzkopf). The divide-and-conquer algorithm is given in Joseph O'Rourke's, "Computational Geometry in C." O'Rourke's book is also a good source for information about orientations and some other geometric primitives.

Convexity: Let us consider a fundamental structure in computational geometry, called the *convex hull*. We will give a more formal definition later, but, given a set P of points in the plane, the convex hull of P , denoted $\text{conv}(P)$, can be defined intuitively by surrounding a collection of points with a rubber band and then letting the rubber band "snap" tightly around the points (see Fig. 1).

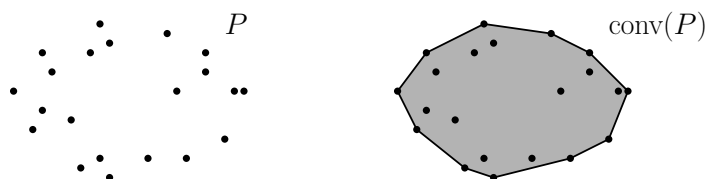


Figure 1: A point set and its convex hull.

There are a number of reasons that the convex hull of a point set is an important geometric structure. One is that it is one of the simplest shape approximations for a set of points. (Other examples include minimum area enclosing rectangles, circles, and ellipses.) It can also be used for approximating more complex shapes. For example, the convex hull of a polygon in the plane or polyhedron in 3-space is the convex hull of its vertices.

Also many algorithms compute the convex hull as an initial stage in their execution or to filter out irrelevant points. For example, the *diameter* of a point set is the maximum distance between any two points of the set. It can be shown that the pair of points determining the diameter are both vertices of the convex hull. Also observe that minimum enclosing convex shapes (such as the minimum area rectangle, circle, and ellipse) depend only on the points of the convex hull.

Let's begin with a few basic definitions regarding convexity and convex sets.

Convexity: A set K is *convex* if given any points $p, q \in K$, the line segment $\overline{pq}$ is entirely contained within K (see Fig. 2(a)).

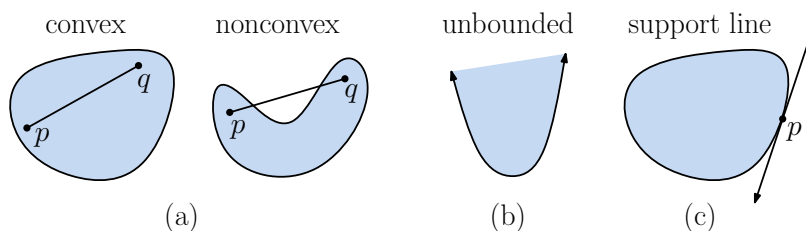


Figure 2: Basic definitions.

Boundedness: A convex body may be bounded, meaning that it can be enclosed within a sphere of a fixed radius or unbounded, meaning that it extends to infinity (see Fig. 2(b)). Examples of unbounded convex sets in the plane include lines, rays, halfplanes, the region lying to one side of a line, and infinite cones. Given a line ℓ , the set of points lying entirely to one side of ℓ (possibly including ℓ itself) is called a *halfplane*.

Support line/hyperplane: An important property of any convex set K in the plane is that at every point p on the boundary of K , there exists at least one line ℓ (or generally a $(d - 1)$ -dimensional hyperplane in higher dimensions) that passes through p such that K lies entirely in one of the closed halfplanes (halfspaces) defined by ℓ (see Fig. 2(c)).

Convex hull: The *convex hull* of any set P is the intersection of all convex sets that contains P , or more intuitively, the smallest convex set that contains P . We will denote this $\text{conv}(P)$.

When computing convex hulls, we will usually take P to be a finite set of points. In such a case, $\text{conv}(P)$ will be a convex polygon. Generally P could be an infinite set of points. For example, we could talk about the convex hull of a collection of circles. The boundary of such a shape would consist of a combination of circular arcs and straight line segments.

Convex Hull Problem: The (planar) *convex hull problem* is, given a set of n points P in the plane, output a representation of P 's convex hull. The convex hull is a closed convex polygon, the simplest representation is a counterclockwise enumeration of the vertices of the convex hull. (Although points of P might lie in the interior of an edge of the boundary of the convex hull, such a point is not considered a vertex. As in many of our algorithms, it will simplify the presentation to avoid lots of special cases by assuming that the points are in *general position*. This is a general concept which effectively means that degenerate configurations (e.g., two points sharing the same x or y coordinate, three points being collinear, etc.) do not arise in the input. It is not difficult to overcome such assumptions, but additional cases must be added, which would complicate the algorithm's description.

Although the output consists only of the boundary of the hull (in fact, only the cyclic sequence of points that make up the vertices of the hull), it is worth keeping in mind that this sequence is *not* the convex hull, but merely a representation thereof. For example, if we were to consider the convex hull problem in 3-dimensional space, the output would be a representation of a convex polyhedron, which would presumably involve a more complex representation, including for example, the vertices, edges, and faces of the hull's boundary.

Graham's scan: We will begin with a presentation of a simple $O(n \log n)$ algorithm for the convex hull problem. It is a simple variation of a famous algorithm for convex hulls, called *Graham's scan*, which dates back to the early 1970's. The algorithm is loosely based on a common approach for building geometric structures called *incremental construction*. In such a algorithm object (points here) are added one at a time, and the structure (convex hull here) is updated with each new insertion.

An important issue with incremental algorithms is the order of insertion. If we were to add points in some arbitrary order, we would need some method of testing whether the newly added point is inside the existing hull. It will simplify things to add points in some appropriately sorted order, in our case, in increasing order of x -coordinate. This guarantees that each newly added point is outside the current hull. (Note that Graham's original algorithm sorted points in a different way. It found the lowest point in the data set and then sorted points cyclically around this point. Sorting by x -coordinate seems to be a bit easier to implement, however.)

Since we are working from left to right, it would be convenient if the convex hull vertices were also ordered from left to right. As mentioned above, the convex hull is a convex polygon, which can be represented as a cyclic sequence of vertices. It will make matters a bit simpler for us to represent the boundary of the convex hull as two polygonal chains, one representing its upper part, called the *upper hull* and one representing the lower part, called the *lower hull* (see Fig. 3(a)).

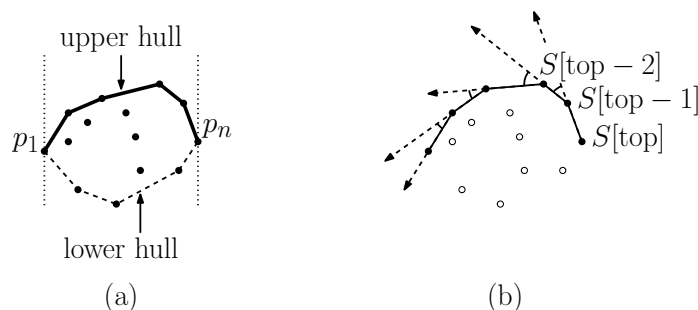


Figure 3: Upper and lower hulls.

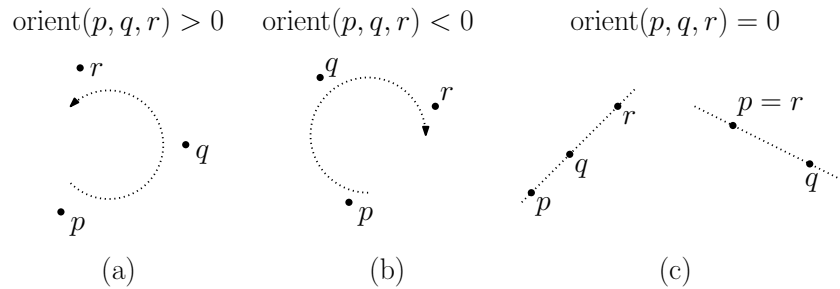
Observe that a point $p \in P$ lies on the upper hull if and only if there is a support line passing through p such that all the points of P lie on or below this line. Similarly, each vertex of the lower hull has a support line such that all the points of P lie on or above this line. The leftmost and rightmost vertices of the convex hull (that is, the two vertices having vertical support lines) are common to the two hulls. (By general position, we may assume there are no duplicate x -coordinates, and so there will be a unique leftmost point and unique rightmost points.) After building both, the two hulls can be concatenated into a single cyclic counterclockwise list.

It suffices to show how to compute the upper hull, since the lower hull is symmetrical. Recall that the points of P are first sorted in increasing order of their x -coordinates, and they will be added one-by-one. Let $\langle p_1, \dots, p_n \rangle$ denote this sorted sequence. For i ranging from 1 to n , we will store the vertices of the current upper hull on a stack S , where the top of the stack corresponds to the most recently added point of P , and the bottom of the stack will be p_1 . Let $S[\text{top}]$ denote the current top of the stack, and let $S[\text{top} - 1]$ denote the element immediately below it.

Observe that as we read the stack elements from top to bottom (that is, from right to left) consecutive triples of points of the upper hull will make a (strict) “left-hand turn” (see Fig. 3(b)). As we push new points on the stack, we will enforce this property by popping points off of the stack that violate it.

Turning and orientations (Aside): Before proceeding with the presentation of the algorithm, we should first make a short digression to discuss the meaning of “left-hand turn.” Given an ordered triple of points $\langle p, q, r \rangle$ in the plane, we say that they have *positive orientation* if they define a counterclockwise oriented triangle (see Fig. 4(a)), *negative orientation* if they define a clockwise oriented triangle (see Fig. 4(b)), and *zero orientation* if they are collinear, which includes as well the case where two or more of the points are identical (see Fig. 4(c)). Note that orientation depends on the order in which the points are given.

Orientation is formally defined as the sign of the determinant of the points given in homogeneous

Figure 4: Orientations of the ordered triple (p, q, r) .

coordinates, that is, by prepending a 1 to each coordinate. For example, in the plane, we define

$$\text{Orient}(p, q, r) = \det \begin{pmatrix} 1 & p_x & p_y \\ 1 & q_x & q_y \\ 1 & r_x & r_y \end{pmatrix}.$$

Observe that in the 1-dimensional case, $\text{Orient}(p, q)$ is just $q - p$. Hence it is positive if $p < q$, zero if $p = q$, and negative if $p > q$. Thus orientation generalizes the familiar 1-dimensional binary relations $<$, $=$, $>$. Observe that the sign of the orientation of an ordered triple is unchanged if the points are translated, rotated, or scaled (by a positive scale factor). A reflection transformation (e.g., $f(x, y) = (-x, y)$) reverses the sign of the orientation. In general, applying any affine transformation to the point alters the *sign* of the orientation according to the *sign* of the determinant of the matrix used in the transformation. (By the way, the notion of orientation can be generalized to $d + 1$ points in d -dimensional space, and is related to the notion of *chirality* in Chemistry and Physics.)

Given a sequence of three points p, q, r , we say that the sequence $\langle p, q, r \rangle$ makes a (strict) *left-hand turn* if $\text{Orient}(p, q, r) > 0$.

Graham's algorithm continued: Returning to the algorithm, let us consider the insertion of the i th point, p_i (see Fig. 5(a)). If the triple $\langle p_i, S[\text{top}], S[\text{top} - 1] \rangle$ forms a strict left-hand turn, then we can simply push p_i onto the stack. Otherwise, we can infer that the middle point of the triple, that is, $S[\text{top}]$, cannot be on the upper hull, and so we pop it off the stack. We repeat this until either reaching a positively oriented triple (see Fig. 5(b)), or until we find that there are fewer than two elements on the stack. We then push p_i on top of the stack, making it the right-most vertex on the upper hull (see Fig. 5(c)). The algorithm is presented in the code block below.

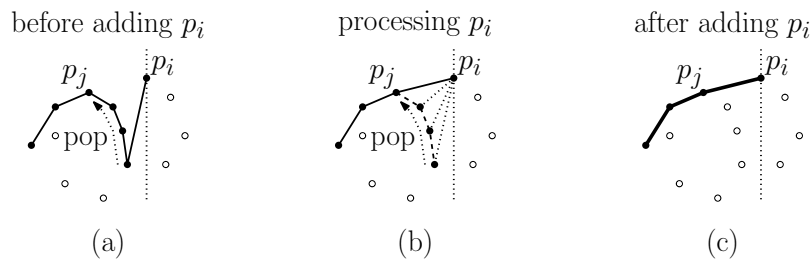


Figure 5: Graham's scan.

Graham's Scan

- (1) Sort the points according to increasing order of their x -coordinates, denoted $\langle p_1, p_2, \dots, p_n \rangle$.
- (2) push p_1 and then p_2 onto S .
- (3) for $i \leftarrow 3, \dots, n$ do:
 - (a) while ($|S| \geq 2$ and $\text{Orient}(p_i, S[\text{top}], S[\text{top} - 1]) \leq 0$) pop S .
 - (b) push p_i onto S .

Correctness: Why is Graham's algorithm correct? As with any incremental algorithm, we need to establish the invariant that holds after each iteration of the loop. Let P_i denote the subsequence consisting of the first i points. Correctness is a direct consequence of the following claim.

Claim: After the insertion of the point p_i , the contents of S (from top to bottom) consist of the vertices of the upper hull of P_i in right to left order.

Proof: The proof is by induction on i . For the basis, observe that on the first entry to the loop we have only two points, p_1 and p_2 , and clearly they must both be on the upper hull of P_2 . Assuming inductively that the stack contents contain the upper hull of P_{i-1} on entry to the loop, we will show that it contains the upper hull of P_i on exit.

First, observe that since p_i is the rightmost point of P_i , it is the last vertex of the upper hull of P_i . Let p_j denote the vertex that immediately precedes p_i on the upper hull of P_i . (Equivalently, if we draw a tangent line through p_i to the upper hull, p_j is the vertex hit by this line.) We will prove the following facts: (a) on entry to the loop body, p_j is on the stack, (b) every point lying above p_j on the stack will be popped off, and (c) the popping process will stop at p_j . From these three facts, it follows that after step (3a), p_j will be on the top of the stack, and after step (3b), p_i will be pushed. Thus, on exit the stack contains the upper hull of P_i , as desired. It remains to prove these three facts.

To prove (a), we observe that by induction p_j is on the stack if and only if it is on the upper hull of P_{i-1} , and recall that this is true if and only if there exists a support line passing through p_j such that all the points of P_{i-1} lie below this line. Clearly, the line $\ell = \overline{p_j p_i}$ is such a line (see Fig. 6(a)).

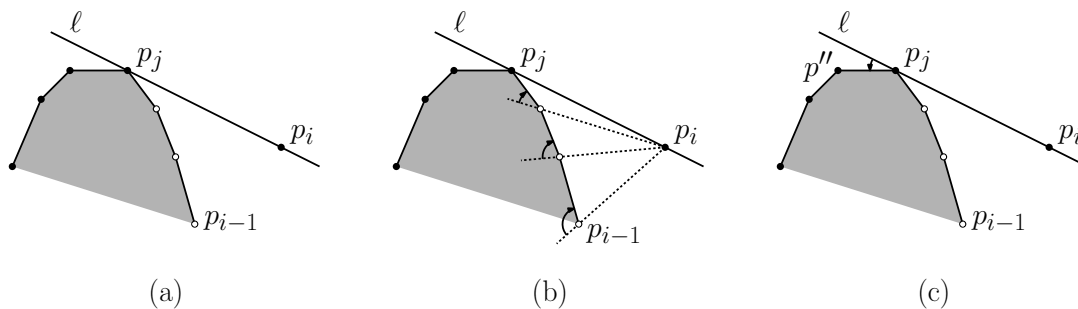


Figure 6: Correctness of Graham's scan.

To prove (b) and (c), observe that for each vertex p_k , for $j < k < i$, that lies on the upper hull of P_{i-1} , the orientation of p_i, p_k , and p_k 's predecessor will be negative (see Fig. 6(b)), but at the

point of tangency p_j the orientation switches to positive (see Fig. 6(c)). Therefore, all of these points will be popped, leaving p_j on the top of the stack.

The lower hull can be computed by a symmetrical algorithm. Once the two hulls are computed, we simply concatenate the two hulls into a single circular list.

Running-time analysis: We will show that Graham's algorithm runs in $O(n \log n)$ time. Clearly, it takes this much time for the initial sorting of the points. After this, we will show that $O(n)$ time suffices for the rest of the computation.

Let d_i denote the number of points that are popped (deleted) on processing p_i . Because each orientation test takes $O(1)$ time, the amount of time spent processing p_i is $O(d_i + 1)$. (The extra $+1$ is for the last point tested, which is not deleted.) Thus, the total running time is proportional to

$$\sum_{i=1}^n (d_i + 1) = n + \sum_{i=1}^n d_i.$$

To bound $\sum_i d_i$, observe that each of the n points is pushed onto the stack once. Once a point is deleted it can never be deleted again. Since each of n points can be deleted at most once, $\sum_i d_i \leq n$. Thus after sorting, the total running time is $O(n)$. Since this is true for the lower hull as well, the total time is $O(2n) = O(n)$.

Convex Hull by Divide-and-Conquer: As with sorting, there are many different approaches to solving the convex hull problem for a planar point set P . Next, we will consider another $O(n \log n)$ algorithm, which is based on divide-and-conquer. It can be viewed as a generalization of the well-known Merge-Sort sorting algorithm (see any standard algorithms text). Here is an outline of the algorithm. As with Graham's scan, we will focus just on computing the upper hull, and the lower hull will be computed symmetrically.

The algorithm begins by sorting the points by their x -coordinate, in $O(n \log n)$ time. It splits the point set in half at its median x -coordinate, computes the upper hulls of the left and right sets recursively, and then merges the two upper hulls into a single upper hull. This latter process involves computing a line, called the *upper tangent*, that is a line of support for both hulls. The remainder of the algorithm is shown in the code section below.

Divide-and-Conquer (Upper) Convex Hull

- (1) If $|P| \leq 3$, then compute the upper hull by brute force in $O(1)$ time and return.
 - (2) Otherwise, partition the point set P into two sets P' and P'' of roughly equal sizes by a vertical line.
 - (3) Recursively compute upper convex hulls of P' and P'' , denoted H' and H'' , respectively (see Fig. 7(a)).
 - (4) Compute the upper tangent $\ell = \overline{p'p''}$ (see Fig. 7(b)).
 - (5) Merge the two hulls into a single upper hull by discarding all the vertices of H' to the right of p' and the vertices of H'' to the left of p'' (see Fig. 7(c)).
-

The only nontrivial step is that of computing the common tangent line between the two upper hulls. Our algorithm will exploit the fact that the two hulls are separated by a vertical line. The algorithm operates by a simple "walking procedure." We initialize p' to be the rightmost point of H' and p'' to be the leftmost point of H'' (see Fig. 8(a)). We will walk p' backwards along H' and walk p''

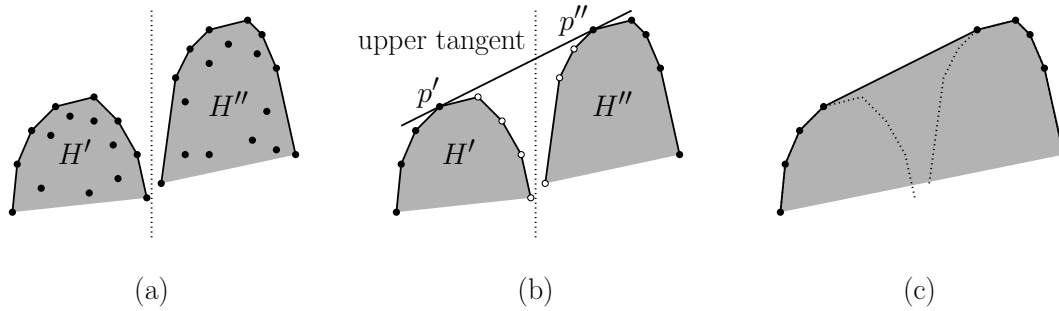


Figure 7: Divide and conquer (upper) convex hull algorithm.

forwards along H'' until we hit the vertices that define the tangent line. As in Graham's scan, it is possible to determine just how far to walk simply by applying orientation tests. In particular, let q' be the point immediately preceding p' on H' , and let q'' be the point immediately following p'' on H'' . Observe that if $\text{Orient}(p', p'', q'') \geq 0$, then we can advance p'' to the next point along H'' (see Fig. 8(a)). Symmetrically, if $\text{Orient}(p'', p', q') \leq 0$, then we can advance p' to its predecessor along H' (see Fig. 8(b)). When neither of these conditions applies, that is, $\text{Orient}(p', p'', q'') < 0$ and $\text{Orient}(p'', p', q') > 0$, we have arrived at the desired points of mutual tangency (see Fig. 8(c)).

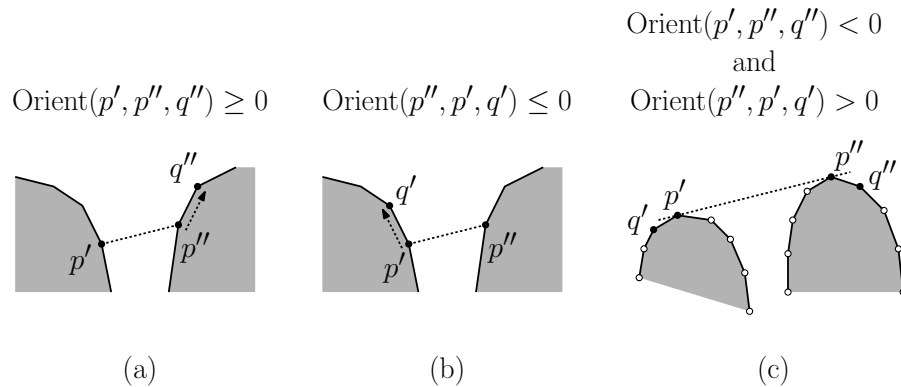


Figure 8: Computing the upper tangent.

There is one rather messy detail in implementing this algorithm. This arises if either q' or q'' does not exist because we have arrived at the leftmost vertex of H' or the rightmost vertex of H'' . We can avoid having to check for these conditions by creating two *sentinel points*. We create a new leftmost vertex for H' that lies infinitely below its original leftmost vertex, and we create a new rightmost vertex for H'' that lies infinitely below its original rightmost vertex. The tangency computation will never arrive at these points, and so we do not need to add a special test for the case when q' and q'' do not exist. The algorithm is presented in the following code block.

A formal proof of correctness of this procedure is similar to that of Graham's scan (but observe that there are now two tangency conditions to be satisfied, not just one). We will leave it as an exercise. Observe that the running time is $O(n)$, because with each step we spend $O(1)$ time and eliminate a point either from H' or from H'' as a candidate for the tangency points, and there are at most n points that can be so eliminated.

UpperTangent(H', H'') :

- (1) Let p' be the rightmost point of H' , and let q' be its predecessor.
 - (2) Let p'' be the leftmost point of H'' , and let q'' be its successor.
 - (3) Repeat the following until $\text{Orient}(p', p'', q'') < 0$ and $\text{Orient}(p'', p', q') > 0$:
 - (a) while ($\text{Orient}(p', p'', q'') \geq 0$) advance p'' and q'' to their successors on H'' .
 - (b) while ($\text{Orient}(p'', p', q') \leq 0$) advance p' and q' to their predecessors on H' .
 - (4) return (p', p'') .
-

Running-time analysis: The asymptotic running time of the algorithm can be expressed by a recurrence. Given an input of size n , consider the time needed to perform all the parts of the procedure, ignoring the recursive calls. This includes the time to partition the point set, compute the upper tangent line, and return the final result. Clearly, each of these can be performed in $O(n)$ time, assuming any standard list representation of the hull vertices. Thus, ignoring constant factors, we can describe the running time by the following recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n \leq 3 \\ n + 2T(n/2) & \text{otherwise.} \end{cases}$$

This is the same recurrence that arises in Mergesort. It is easy to show that it solves to $T(n) \in O(n \log n)$ (see any standard algorithms text).